HUAWEI AI CERTIFICATION TRAINING

HCIA-AI

Guía de Laboratorio de Machine Learning

EDICIÓN: 4.0

HUAWEI

HUAWEI TECHNOLOGIES CO., LTD

Copyright © Huawei Technologies Co., Ltd. 2025. Todos los derechos reservados.
Ninguna parte de este documento puede ser reproducida o transmitida en ninguna forma ni por ningún medio sin el consentimiento previo por escrito de Huawei Technologies Co., Ltd.

Marcas Comerciales y Permisos
HUAWEI y otras marcas comerciales de Huawei son marcas registradas de Huawei Technologies Co., Ltd.
Todas las demás marcas comerciales y nombres comerciales mencionados en este documento son propiedad de sus respectivos titulares.

Aviso
Los productos, servicios y funciones adquiridos están estipulados por el contrato celebrado entre Huawei y el cliente. La totalidad o parte de los productos, servicios y funciones descritos en este documento pueden no estar dentro del alcance de la compra o del alcance de uso. A menos que se especifique lo contrario en el contrato, todas las declaraciones, información y recomendaciones en este documento se proporcionan "TAL COMO ESTÁN" sin garantías, representaciones ni condiciones de ningún tipo.
La información en este documento está sujeta a cambios sin previo aviso.

Dirección: Huawei Industrial Base Bantian, Longgang Shenzhen 518129, República Popular de China

Sitio web: https://e.huawei.com

Sistema de Certificación Huawei

La Certificación Huawei es parte integral de la estrategia Platform + Ecosystem de la empresa. Soporta la nueva infraestructura de Tecnologías de la Información y Comunicación (TIC) que combina Cloud-Pipe-Device, y abarca todos los dominios TIC, incluyendo certificación para la arquitectura TIC y sus aplicaciones, así como certificación para servicios y plataformas en la nube.

El sistema de certificación ofrece tres niveles: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP) y Huawei Certified ICT Expert (HCIE), para satisfacer los requisitos de aprendizaje y avance de los profesionales TIC.

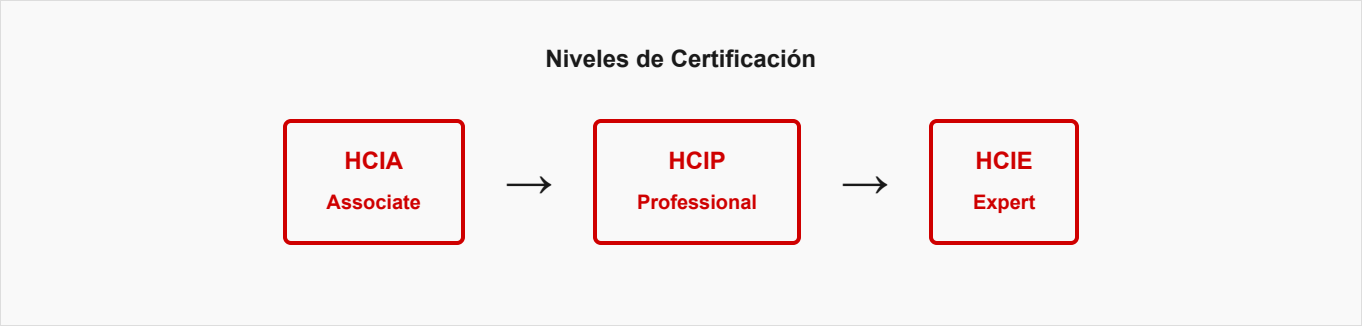
Las certificaciones Huawei cubren todos los campos TIC y se adaptan a la tendencia de convergencia del sector. Con su sistema líder de desarrollo de talento y estándares de certificación, Huawei se compromete a fomentar nuevo talento TIC en la era digital, construyendo un ecosistema saludable de talento TIC.

El sistema de certificación HCIA-AI V4.0 está diseñado para capacitar y certificar ingenieros capaces de diseñar, desarrollar e innovar productos y soluciones de IA mediante algoritmos de machine learning y deep learning.

Con la certificación HCIA-AI V4.0, estás preparado para:

- Comprender la historia del desarrollo de la IA.
- Dominar algoritmos relacionados con machine learning tradicional y deep learning.
- Ser capaz de construir, entrenar y desplegar redes neuronales usando el framework de desarrollo PyTorch.
- Ser competente en ventas, marketing, gestión de productos, gestión de proyectos, soporte técnico y otras posiciones de IA.

El sistema de certificación Huawei te introduce a la industria y el mercado, te ayuda a innovar y te permite mantenerte a la vanguardia de la IA.



Acerca de Este Documento

Descripción General

Este documento está dirigido a candidatos que se preparan para el examen HCIA-AI y a otros que desean aprender conocimientos básicos de machine learning.

Descripción

Esta guía de laboratorio comprende los siguientes cinco experimentos:

- **Experimento 1 — Regresión Lineal.** Se usa el paquete Python scikit-learn para implementar un algoritmo simple de regresión lineal.
- **Experimento 2 — Regresión Lineal Expandida.** Se usa el paquete básico numpy para implementar algoritmos como regresión lineal y descenso de gradiente desde cero.
- **Experimento 3 — Regresión Logística.** Se usa el algoritmo de regresión logística del paquete de herramientas para implementar una clasificación simple.
- **Experimento 4 — Árbol de Decisión.** Los participantes construirán un árbol de decisión para predecir el clima y visualizarlo.
- **Experimento 5 — Algoritmo K-means de Agrupamiento.**

Conocimientos Previos Requeridos

Este curso es para la certificación profesional de Huawei. Se espera que los participantes cumplan con los siguientes requisitos:

- Tener conocimientos básicos de programación en Python.
- Tener habilidades matemáticas básicas, incluyendo álgebra lineal y teoría de probabilidad.

Descripción del Entorno de Laboratorio

El entorno de laboratorio es Python 3.7. Para más detalles sobre cómo configurar el entorno, consulta la Guía de Configuración del Entorno.

Los conjuntos de datos usados en este laboratorio se pueden obtener en:
`https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V4.0/Dataset.zip`

Contenido

Acerca de Este Documento	**3**
Descripción General	3
Descripción	3
Conocimientos Previos Requeridos	3
Descripción del Entorno de Laboratorio	3
1 Implementación de Algoritmos Comunes de Machine Learning	**5**
1.1 Introducción	5
1.1.1 Acerca de Este Laboratorio	5
1.1.2 Objetivos	5
1.2 Implementación del Código	5
1.2.1 Regresión Lineal	5
1.2.2 Implementación de Regresión Lineal (Experimento Expandido)	7
1.2.3 Regresión Logística	10
1.2.4 Árbol de Decisión	12
1.2.5 Implementación del Algoritmo K-means	14
1.3 Preguntas	19

1 Implementación de Algoritmos Comunes de Machine Learning

1.1 Introducción

1.1.1 Acerca de Este Laboratorio

Este laboratorio introduce algoritmos comunes de machine learning para ayudarte a comprender mejor sus funciones y usos. En concreto, explicará cómo construir un algoritmo de regresión lineal desde cero e implementar el árbol de decisión y el agrupamiento K-means con scikit-learn.

1.1.2 Objetivos

- Construir un algoritmo de regresión lineal desde cero.
- Dominar el uso de algoritmos de clasificación y regresión.
- Dominar el uso del algoritmo de agrupamiento K-means.
- Dominar el proceso de implementación de algoritmos de machine learning.

1.2 Implementación del Código

1.2.1 Regresión Lineal

Paso 1 Importar los paquetes dependientes.

ENTRADA:

```
from sklearn.linear_model import LinearRegression  # Importar el modelo de regresión
lineal
import matplotlib.pyplot as plt  # La librería de graficación
import numpy as np
```

Paso 2 Construir y visualizar un conjunto de datos de precios de casas.

ENTRADA:

```
x = np.array([121, 125, 131, 141, 152, 161]).reshape(-1,1)  # x denota el área de la casa
como característica
y = np.array([300, 350, 425, 405, 496, 517])  # y denota el precio de la casa
plt.scatter(x, y)
plt.xlabel("area")   # El eje X indica el área
plt.ylabel("price")  # El eje Y indica el precio
plt.show()
```

SALIDA:

```
[Gráfico de dispersión: área (eje X, 120-165) vs precio (eje Y, 300-525)]
```

Paso 3 Entrenar el modelo.

ENTRADA:

```
lr = LinearRegression()  # Encapsular el modelo de regresión lineal en un objeto
lr.fit(x, y)             # Entrenar el modelo con el conjunto de datos
```

Paso 4 Visualizar el modelo.

ENTRADA:

```
w = lr.coef_       # Pendiente del modelo
b = lr.intercept_  # Intercepto del modelo
print('Pendiente:', w)
print('Intercepto:', b)
```

SALIDA:

```
Pendiente: [4.98467124]
Intercepto: -274.8769665187576
```

ENTRADA:

```
plt.scatter(x, y)
plt.xlabel("area")   # El eje X indica el área
plt.ylabel("price")  # El eje Y indica el precio
plt.plot([x[0], x[-1]], [x[0]*w+b, x[-1]*w+b])
plt.show()
```

SALIDA:

```
[Gráfico: nube de puntos con línea de regresión ajustada]
```

Paso 5 Iniciar una tarea de predicción usando el modelo.

ENTRADA:

```
testX = np.array([[130]])  # Una muestra de prueba con área 130
lr.predict(testX)
```

SALIDA:

```
Array([373.13029447])  # El modelo predice el precio de la casa para la muestra
```


1.2.2 Implementación de Regresión Lineal (Experimento Expandido)

Este experimento usa el conjunto de datos **lr2_data.txt**, que contiene datos simulados de área y precio de casas. Para obtener el conjunto de datos, consulta la Guía de Configuración del Entorno de Laboratorio.

Paso 1 Importar dependencias.

ENTRADA:

```
import numpy as np
import matplotlib.pyplot as plt
```

Paso 2 Definir la función para calcular gradientes.

ENTRADA:

```
def generate_gradient(X, theta, y):
    sample_count = X.shape[0]
    # Calcular el gradiente basado en la matriz 1/m Σ(((h(x^i)-y^i)) x_j^i)
    return (1./sample_count) * X.T.dot(X.dot(theta) - y)
```

Paso 3 Definir la función para leer conjuntos de datos.

ENTRADA:

```
def get_training_data(file_path):
    orig_data = np.loadtxt(file_path, skiprows=1)  # Ignorar el título en la primera fila
    cols = orig_data.shape[1]
    return (orig_data, orig_data[:, :cols - 1], orig_data[:, cols-1:])
```

Paso 4 Definir la función para inicializar parámetros.

ENTRADA:

```
# Inicializar el arreglo θ
def init_theta(feature_count):
```

```
    return np.ones(feature_count).reshape(feature_count, 1)
```

Paso 5 Definir la función para implementar el descenso de gradiente.

ENTRADA:

```
def gradient_descending(X, y, theta, alpha):
    Jthetas = []  # Registrar la tendencia de cambio de la función de costo J(θ)

    # Calcular la función de pérdida: (y^i - h(x^i))^2
    Jtheta = (X.dot(theta) - y).T.dot(X.dot(theta) - y)
    index = 0
    gradient = generate_gradient(X, theta, y)  # Calcular el gradiente

    while not np.all(np.absolute(gradient) <= 1e-5):  # Terminar cuando el gradiente sea
< 0.00001
        theta = theta - alpha * gradient
        gradient = generate_gradient(X, theta, y)
        # Calcular la función de pérdida: (y^i - h(x^i))^2
        Jtheta = (X.dot(theta) - y).T.dot(X.dot(theta) - y)
        if (index + 1) % 10 == 0:
            Jthetas.append((index, Jtheta[0][0]))  # Registrar el resultado cada 10
cálculos
        index += 1
    return theta, Jthetas
```

Paso 6 Definir la función para visualizar la curva de cambio de la función de pérdida.

ENTRADA:

```
# Graficar la curva de cambio de la función de pérdida
def showJTheta(diff_value):
    p_x = []
    p_y = []
    for (index, sum) in diff_value:
        p_x.append(index)
        p_y.append(sum)
    plt.plot(p_x, p_y, color='b')
    plt.xlabel('pasos')
    plt.ylabel('función de pérdida')
    plt.title('curva paso - función de pérdida')
    plt.show()
```

Paso 7 Definir la función para visualizar puntos de datos y la curva ajustada.

ENTRADA:

```
# Graficar los puntos de datos reales y la curva ajustada
def showlinercurve(theta, sample_training_set):
    x, y = sample_training_set[:, 1], sample_training_set[:, 2]
```

```
    z = theta[0] + theta[1] * x
    plt.scatter(x, y, color='b', marker='x', label="datos de muestra")
    plt.plot(x, z, 'r', color='r', label="curva de regresión")
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('curva de regresión lineal')
    plt.legend()
    plt.show()
```

Paso 8 Graficar los resultados finales.

ENTRADA:

```
# Leer el conjunto de datos
training_data_include_y, training_x, y = get_training_data("./ML/02/lr2_data.txt")
# Obtener el número de muestras y características
sample_count, feature_count = training_x.shape
# Definir el paso de aprendizaje α
alpha = 0.01
# Inicializar θ
theta = init_theta(feature_count)
# Obtener el parámetro final θ y el costo
result_theta, Jthetas = gradient_descending(training_x, y, theta, alpha)
# Mostrar el parámetro
print("w:{}".format(result_theta[0][0]), "b:{}".format(result_theta[1][0]))
showJTheta(Jthetas)
showlinercurve(result_theta, training_data_include_y)
```

SALIDA (IGNORAR ADVERTENCIAS):

```
w:3.0076279423997594  b:1.668677412281192
```

```
[Gráfico 1: Curva paso-función de pérdida: baja rápidamente de ~800 a ~0 en los primeros
pasos]
[Gráfico 2: Curva de regresión lineal ajustada sobre los datos de muestra]
```


1.2.3 Regresión Logística

Este experimento usa un conjunto de datos personalizado de renta y área de casas. El conjunto de datos se define durante la fase inicial del experimento.

Paso 1 Importar dependencias.

ENTRADA:

```
# Importar StandardScaler desde sklearn.preprocessing
from sklearn.preprocessing import StandardScaler
# Importar LogisticRegression desde sklearn.linear_model
from sklearn.linear_model import LogisticRegression
```

Paso 2 Definir el conjunto de datos.

ENTRADA:

```
# Cada elemento en X denota la renta y el área
# y indica si se alquila la habitación (0: no; 1: sí)
X = [[2200,15],[2750,20],[5000,40],[4000,20],[3300,20],[2000,10],[2500,12],[12000,80],
     [2880,10],[2300,15],[1500,10],[3000,8],[2000,14],[2000,10],[2150,8],[3400,20],
     [5000,20],[4000,10],[3300,15],[2000,12],[2500,14],[10000,100],[3150,10],
     [2950,15],[1500,5],[3000,18],[8000,12],[2220,14],[6000,100],[3050,10]]

y = [1,1,0,0,1,1,1,0,1,1,0,1,1,0,0,0,1,1,1,0,1,0,0,1,2,0,1,1,0]
```

Paso 3 Preprocesar los datos.

Estandarizar los datos para garantizar que la varianza de los datos de características en cada dimensión sea 1 y la media sea 0. De esta manera, el resultado de la predicción no es dominado por valores de características grandes de algunas dimensiones.

ENTRADA:

```
ss = StandardScaler()
X_train = ss.fit_transform(X)
```

MOSTRAR LOS DATOS ESTANDARIZADOS:

```
print(X_train)
```

SALIDA:

```
[[-0.60583897 -0.29313058]
 [-0.37682768 -0.09050576]
 [ 0.56003671  0.71999355]
 [ 0.14365254 -0.09050576]
 ...
 [-0.25191242 -0.49575541]]
```

Paso 4 Ajustar los datos.

ENTRADA:

```
# Usar el método fit de LogisticRegression para entrenar los parámetros del modelo
lr = LogisticRegression()
lr.fit(X_train, y)
```

SALIDA:

```
LogisticRegression()
```

Paso 5 Predecir los datos.

ENTRADA:

```
testX = [[2000, 8]]
X_test = ss.transform(testX)
print("Valor a predecir: ", X_test)
label = lr.predict(X_test)
print("etiqueta predicha = ", label)
# Mostrar la probabilidad predicha
prob = lr.predict_proba(X_test)
print("probabilidad = ", prob)
```

SALIDA:

```
Valor a predecir: [[-0.68911581 -0.57680534]]
etiqueta predicha =  [1]
probabilidad =  [[0.41886952 0.58113048]]
```


1.2.4 Árbol de Decisión

Este experimento usa el conjunto de datos **tennis.txt**, que contiene 14 muestras. Cada muestra contiene características relacionadas con el clima e indica si es adecuado para jugar tenis.

Paso 1 Importar dependencias.

ENTRADA:

```
import pandas as pd
import numpy as np
from sklearn import tree
import pydotplus
```

Paso 2 Definir la función para generar un árbol de decisión.

ENTRADA:

```
# Generar un árbol de decisión
def createTree(trainingData):
    data = trainingData.iloc[:, :-1]    # Matriz de características
    labels = trainingData.iloc[:, -1]   # Etiquetas
```

```
    trainedTree = tree.DecisionTreeClassifier(criterion="entropy")  # Clasificador de
árbol de decisión
    trainedTree.fit(data, labels)        # Entrenar el modelo
    return trainedTree
```

Paso 3 Definir la función para guardar el diagrama del árbol generado.

ENTRADA:

```
def showtree2pdf(trainedTree, finename):
    dot_data = tree.export_graphviz(trainedTree, out_file=None)  # Exportar en formato
Graphviz
    graph = pydotplus.graph_from_dot_data(dot_data)
    graph.write_pdf(finename)  # Guardar el diagrama del árbol en formato PDF
```

Paso 4 Definir la función para generar datos vectorizados.

En la función, **pd.Categorical(list).codes** obtiene la lista de números de secuencia correspondientes a los datos originales, para convertir la información categórica en información numérica.

ENTRADA:

```
def data2vectoc(data):
    names = data.columns[:-1]
    for i in names:
        col = pd.Categorical(data[i])
        data[i] = col.codes
    return data
```

Paso 5 Invocar la función para predicción.

ENTRADA:

```
data = pd.read_table("./ML/tennis.txt", header=None, sep='\t')  # Leer datos de
entrenamiento
trainingvec = data2vectoc(data)    # Vectorizar datos
decisionTree = createTree(trainingvec)   # Crear árbol de decisión
showtree2pdf(decisionTree, "tennis.pdf") # Graficar el árbol de decisión
```

Se genera un diagrama del árbol de decisión llamado **tennis.pdf** en la máquina local.

> **Interpretación del árbol:**
> • **X[2]** = tercera variable de característica (humedad)
> • **X[0]** = primera variable de característica (clima)
> • **X[3]** = cuarta variable de característica (viento)
> • **entropy** = valor de entropía del nodo
> • **samples** = número de muestras (ej. 14 en el nodo raíz)
> • **value** = número de muestras de cada tipo (ej. [5,9] = 5 "No" y 9 "Sí")

PREDECIR UNA NUEVA MUESTRA. ENTRADA:

```
testVec = [0, 0, 1, 1]  # Clima: soleado, temperatura: baja, humedad: alta, viento:
fuerte
print(decisionTree.predict(np.array(testVec).reshape(1, -1)))  # Predecir
```

SALIDA:

```
['N']
```


1.2.5 Implementación del Algoritmo K-means

Paso 1 Importar dependencias.

make_blobs se usa para generar el conjunto de datos requerido para este experimento, **matplotlib** se usa para la visualización de datos, y **KMeans** se usa para ajustar los datos basándose en el algoritmo K-means.

ENTRADA:

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

Paso 2 Generar el conjunto de datos.

make_blobs() se usa para generar un conjunto de datos para una tarea de agrupamiento, devolviendo el conjunto de datos generado y las etiquetas. Nota: el algoritmo K-means a menudo se usa para procesar conjuntos de datos sin etiqueta en producción. El conjunto de datos generado contiene etiquetas solo para facilitar la comprensión.

ENTRADA:

```
X, y = make_blobs(n_samples=500, n_features=2, centers=4, random_state=1)
```

n_samples = número de muestras, **n_features** = número de características de cada muestra (dimensión de los datos), **centers** = número de categorías, **random_state** = semilla para el generador de números aleatorios (garantiza que los datos generados sean siempre iguales).

ENTRADA:

```
Print("Dimensión de X es {}".format(X.shape))
Print("Dimensión de y es {}".format(y.shape))
```

SALIDA:

```
Dimensión de X es (500, 2)
Dimensión de y es (500,)
```

El conjunto de datos contiene 500 puntos de muestra, y cada punto contiene dos características.

Paso 3 Dibujar gráficos de dispersión.

Primero, dibujar directamente un gráfico de dispersión del conjunto de datos. Las etiquetas generadas durante la creación del conjunto de datos no se consideran por ahora. El gráfico contiene cuatro grupos distintos.

ENTRADA:

```
fig, ax1 = plt.subplots(1)
ax1.scatter(X[:, 0], X[:, 1]
            , marker='o'  # Establecer la forma del punto en círculo
            , s=8          # Establecer el tamaño del punto
            )
plt.show()
```

SALIDA:

```
[Gráfico de dispersión sin colores: 4 grupos visibles de puntos]
```

A continuación, dibujar un gráfico de dispersión donde los puntos de diferentes colores corresponden a las etiquetas generadas.

ENTRADA:

```
color = ["red", "pink", "orange", "green"]
fig, ax1 = plt.subplots(1)

for i in range(4):
    ax1.scatter(X[y==i, 0], X[y==i, 1]  # Dibujar el color según la etiqueta
                , marker='o'   # Establecer la forma del punto en círculo
                , s=8          # Establecer el tamaño del punto
                , c=color[i]
                )
plt.show()
```

SALIDA:

```
[Gráfico de dispersión coloreado: 4 grupos en rojo, rosa, naranja y verde]
```

Paso 4 Realizar agrupamiento K-means.

Usar **sklearn.cluster.KMeans** de scikit-learn para implementar agrupamiento K-means. Primero, agrupar las muestras de datos en tres tipos.

ENTRADA:

```
n_clusters = 3
```

```
cluster1 = KMeans(n_clusters=n_clusters, random_state=3).fit(X)
```

VER LA ETIQUETA DE CADA PUNTO DE MUESTRA DESPUÉS DEL AGRUPAMIENTO:

```
y_pred1 = cluster1.labels_
print(y_pred1)
```

SALIDA:

```
[0 0 2 1 2 2 2 0 0 2 1 2 0 2 0 1 2 2 2 0 1 1 2 2 0 1 2 0 2 0 2 ...]
```

IMPRIMIR EL CENTROIDE DE CADA GRUPO:

```
centroid1 = cluster1.cluster_centers_
print(centroid1)
```

SALIDA:

```
[[-7.09306648 -8.10994454]
 [-1.54234022  4.43517599]
 [-8.0862351  -3.5179868 ]]
```

VISUALIZAR EL RESULTADO DEL AGRUPAMIENTO:

```
color = ["red", "pink", "orange", "gray"]

fig, ax1 = plt.subplots(1)

for i in range(n_clusters):
    ax1.scatter(X[y_pred1==i, 0], X[y_pred1==i, 1]
                , marker='o'  # Círculo
                , s=8
                , c=color[i]
                )

ax1.scatter(centroid1[:, 0], centroid1[:, 1]
            , marker="x"
            , s=15
            , c="black")
plt.show()
```

SALIDA:

```
[Gráfico: 3 grupos coloreados con centroides marcados como 'x' negro]
```

El gráfico muestra que las muestras de datos se agrupan en tres clústeres, y el centroide de cada clúster está representado por **x**.

Paso 5 Realizar agrupamiento K-means nuevamente.

A continuación, realizar los pasos anteriores para usar K-means y agrupar las muestras de datos en cuatro tipos.

ENTRADA:

```
n_clusters = 4
cluster2 = KMeans(n_clusters=n_clusters, random_state=0).fit(X)
y_pred2 = cluster2.labels_
centroid2 = cluster2.cluster_centers_
print("Centroide: {}".format(centroid2))
```

SALIDA:

```
Centroide: [[ -6.08459039  -3.17305983]
 [ -1.54234022   4.43517599]
 [ -7.09306648  -8.10994454]
 [-10.00969056  -3.84944007]]
```

VISUALIZAR EL RESULTADO DEL AGRUPAMIENTO:

```
color = ["red", "pink", "orange", "green"]

fig, ax1 = plt.subplots(1)

for i in range(n_clusters):
    ax1.scatter(X[y_pred2==i, 0], X[y_pred2==i, 1]
                , marker='o'
                , s=8
                , c=color[i]
                )

ax1.scatter(centroid2[:, 0], centroid2[:, 1]
            , marker="x"
            , s=15
            , c="black")
plt.show()
```

SALIDA:

```
[Gráfico: 4 grupos coloreados con centroides marcados como 'x' negro]
```


1.3 Preguntas

1. ¿Cuáles son las diferencias entre la regresión lineal y la regresión logística?
2. ¿Por qué es necesario estandarizar los datos antes de aplicar regresión logística?

3. ¿Qué criterios de división se pueden usar en un árbol de decisión? ¿Cuál es la diferencia entre entropía e índice de Gini?
4. ¿Cuál es la influencia del valor de K en el algoritmo K-means? ¿Cómo se elige un valor de K apropiado?
5. ¿Cuáles son las limitaciones del algoritmo K-means?

1.4 Resumen

En este laboratorio aprendiste a:

- **Regresión Lineal:** Usar scikit-learn para construir y entrenar un modelo de regresión lineal, visualizar los datos y hacer predicciones.
- **Regresión Lineal Expandida:** Implementar el algoritmo de descenso de gradiente desde cero usando numpy, comprendiendo la mecánica interna del aprendizaje de parámetros.
- **Regresión Logística:** Aplicar estandarización de datos y usar LogisticRegression de scikit-learn para clasificación binaria.
- **Árbol de Decisión:** Construir un árbol de decisión usando criterio de entropía para clasificación, y visualizarlo como PDF.
- **K-means:** Usar KMeans de scikit-learn para agrupamiento no supervisado, experimentando con diferentes valores de K y visualizando los centroides.

Estos cinco algoritmos son los más importantes del examen HCIA-AI. Dominar su implementación práctica en Python te prepara directamente para las preguntas del examen.

Conjunto de Datos del Laboratorio

Descargar desde:
`https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V4.0/Dataset.zip`

Estructura esperada de carpetas:

```
./ML/
  ├── 02/lr2_data.txt    ← Experimento 2 (Regresión Lineal Expandida)
  └── tennis.txt         ← Experimento 4 (Árbol de Decisión)
```